Contents

SHYFEM Fortran Codebase Analysis **2**
1. Architecture Overview . . . 3
System Architecture Diagram . . . 3
Module Integration Diagram . . . 3
Data Flow Summary . . . 8
Single Timestep Execution Order . . . 8
2. Core Hydrodynamics . . . 9
`shyfem_subs.f90` – Main Simulation Loop (~2,100 lines) . . . 9
`hydrodynamic.f90` – Hydrodynamic Solver (~2,200 lines) . . . 9
`explicit.f90` – Explicit Forcing Terms (~1,322 lines) . . . 10
3. Scalar Transport . . . 10
`concentration.f90` + `concentration_omp.f90` (~3,070 lines combined) . . . 10
4. Turbulence Closure . . . 11
`turbulence.f90` (~1,216 lines) . . . 11
5. Baroclinic (Density) Computations . . . 11
`baroclinic.f90` (~1,072 lines) . . . 11
6. Element and Depth Operations . . . 12
`elems_dealing.f90` (~1,083 lines) . . . 12
7. Physics and Coupling Modules . . . 12
7.1 Parameter System (`src/utils/generic/para.f90`) . . . 12
7.2 Sediment Transport (`sedim_admin.f90`) . . . 13
7.3 Two-Way Sediment-Hydrodynamics Coupling . . . 13
Two-Way Coupling Diagram . . . 13
7.4 Coupling Architecture Diagram . . . 13
7.5 Waves (`waves_admin.f90`) . . . 13
7.6 Wetting and Drying (`wetdry.f90`) . . . 15
7.7 Bottom Friction (`chezy.f90`) . . . 15
7.8 Diffusion (`diffusion_admin.f90`) . . . 15
7.9 Physical Constants (`mod_physical_constants.f90`) . . . 15
8. MPI Parallelization Infrastructure . . . 15
MPI and OpenMP Architecture Diagram . . . 15
8.1 Architecture . . . 15
8.2 Key Data Structures . . . 16
8.3 Element Ownership Encoding (`id_elem`) . . . 16
8.4 Ghost Exchange Kernel (`mpi_node_internal.f90`) . . . 16
8.5 Domain Decomposition Setup . . . 16
8.6 Triple-Point Exchange (`mpi_tripple.f90`) . . . 16
8.7 TVD Scalar Advection MPI (`mpi_tvd.f90`) . . . 17
8.8 MPI File Summary . . . 17
9. Lagrangian Particle Tracking . . . 17
Lagrangian System Diagram . . . 17
9.1 Overview . . . 17
9.2 Advection Engine . . . 18
9.3 Diffusion . . . 18
9.4 Sub-Modules . . . 18
9.5 Release Mechanisms (`lagrange_init.f90`) . . . 18
9.6 MPI Exchange (`lagrange_mpi.f90`, ~497 lines) . . . 18
Particle Data Structure (`lagrange_data.f90`) . . . 19
Serialization Format (13 + ncust doubles per particle) . . . 19
Exchange Protocol (`lgr_mpi_exchange`) . . . 19
Other Key Routines . . . 19
Integration Points . . . 19
10. Ecology and Biogeochemistry . . . 20
Ecology Integration Diagram . . . 20

10.1 Architecture . . . 20
10.2 WEUTRO (Eutrophication Model) . . . 20
10.3 AQUABC (ALUKAS – Advanced Biogeochemistry) . . . 21
10.4 BFM (ERSEM) . . . 21
10.5 Mercury Module . . . 21
10.6 Toxicant Module (atoxi3d) . . . 21
10.7 Water Renewal Time . . . 21
10.8 Timestep Call Order . . . 21
11. Ice Model . . . 23
Ice Model Diagram . . . 23
11.1 Overview . . . 23
11.2 Physics . . . 23
11.3 Coupling Channels . . . 23
11.4 Limitations . . . 23
11.5 Heat Flux Administration (heat_admin2.f90) . . . 23
12. Sediment Transport (Detailed) . . . 24
Sediment System Diagram . . . 24
12.1 SEDTRANS05 Pipeline . . . 24
12.2 SEDTRANS05 Physics Kernel . . . 24
12.3 Transport Branches . . . 24
12.4 Grain Size Classes . . . 24
12.5 Bed Layer Stack . . . 26
12.6 Morphological Acceleration (MORPHO) . . . 26
12.7 Roughness Feedback (DISABLED) . . . 26
12.8 Two-Way Coupling Summary . . . 26
13. Build System . . . 26
Hierarchy . . . 26
Compiler Profiles . . . 26
Special Compilation Flags . . . 27
Key Build Commands . . . 27
14. Solver Infrastructure . . . 27
Available Solvers . . . 27
SPARSKIT (Default) . . . 28
PETSc . . . 28
15. Code Quality Metrics . . . 28
Allocation Error Handling (stat=) . . . 28
Modern File I/O (newunit=) . . . 28
Other Metrics . . . 28
Source File Counts . . . 28
16. Observations and Improvement Opportunities . . . 29
Strengths . . . 29
Known Limitations . . . 29
Improvement Opportunities Ranked by Impact . . . 30
Optimization Status . . . 30
Benchmarks (mar_menor test case, small mesh) . . . 31

SHYFEM Fortran Codebase Analysis

Comprehensive technical analysis of the SHYFEM v8.1.1 Fortran implementation. Generated 2026-03-05.

Table of Contents

- 1. Architecture Overview
  - System Architecture Diagram
  - Module Integration Diagram

  - Data Flow Summary
  - Single Timestep Execution Order
- 2. Core Hydrodynamics
- 3. Scalar Transport
- 4. Turbulence Closure
- 5. Baroclinic (Density) Computations
- 6. Element and Depth Operations
- 7. Physics and Coupling Modules
  - Two-Way Coupling Diagram
- 8. MPI Parallelization Infrastructure
  - MPI and OpenMP Architecture Diagram
- 9. Lagrangian Particle Tracking
  - Lagrangian System Diagram
- 10. Ecology and Biogeochemistry
  - Ecology Integration Diagram
- 11. Ice Model
  - Ice Model Diagram
- 12. Sediment Transport (Detailed)
  - Sediment System Diagram
- 13. Build System
- 14. Solver Infrastructure
- 15. Code Quality Metrics
- 16. Observations and Improvement Opportunities

---

1. Architecture Overview

SHYFEM implements a 3D finite element hydrodynamic model using a semi-implicit time-stepping scheme on unstructured triangular meshes. The codebase is organized into four tiers:

Layer	Location	Responsibility
Program control	`src/shyfem/shyfem_subs.f90`	Main loop, initialization, module coupling
Physics modules	`src/shyfem/*.f90` (~134 files)	Hydrodynamics, transport, turbulence, sediment, waves
Utility libraries	`src/utils/` (~250 files)	MPI, I/O, mesh, parameter system, modules
External solvers	`src/external/` (~85 files)	SPARSKIT, GOTM

Parallelization is hybrid MPI+OpenMP: MPI handles domain decomposition across ranks; OpenMP handles intra-rank element/node-level task parallelism via graph-coloring subsets.

System Architecture Diagram

Module Integration Diagram

The diagram below shows every major module, the data it produces (downward arrows) or consumes (upward arrows), and the file that owns the interaction. Dashed lines denote optional coupling controlled by runtime parameters.

```
+=====================================================================+
|                      PROGRAM CONTROL LAYER                          |
|                      shyfem_subs.f90                                |
|   shyfem_initialize --> shyfem_run (time loop) --> shyfem_finalize  |
+==============+======================+===============================+
```

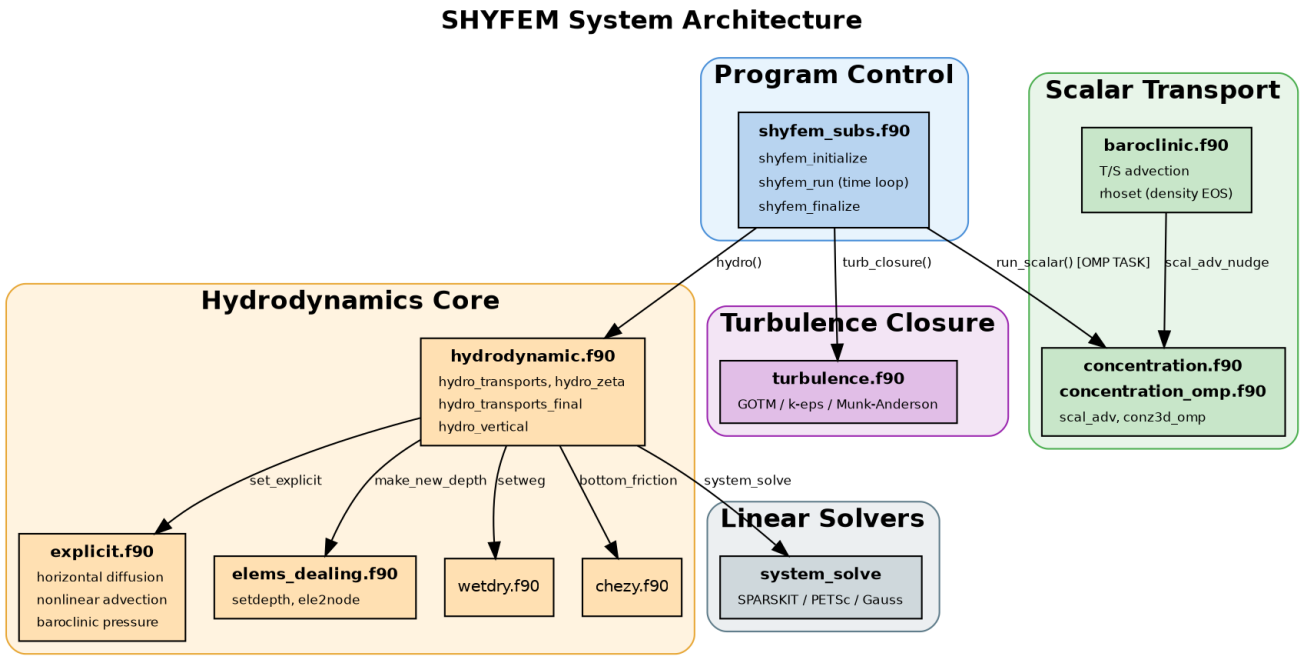
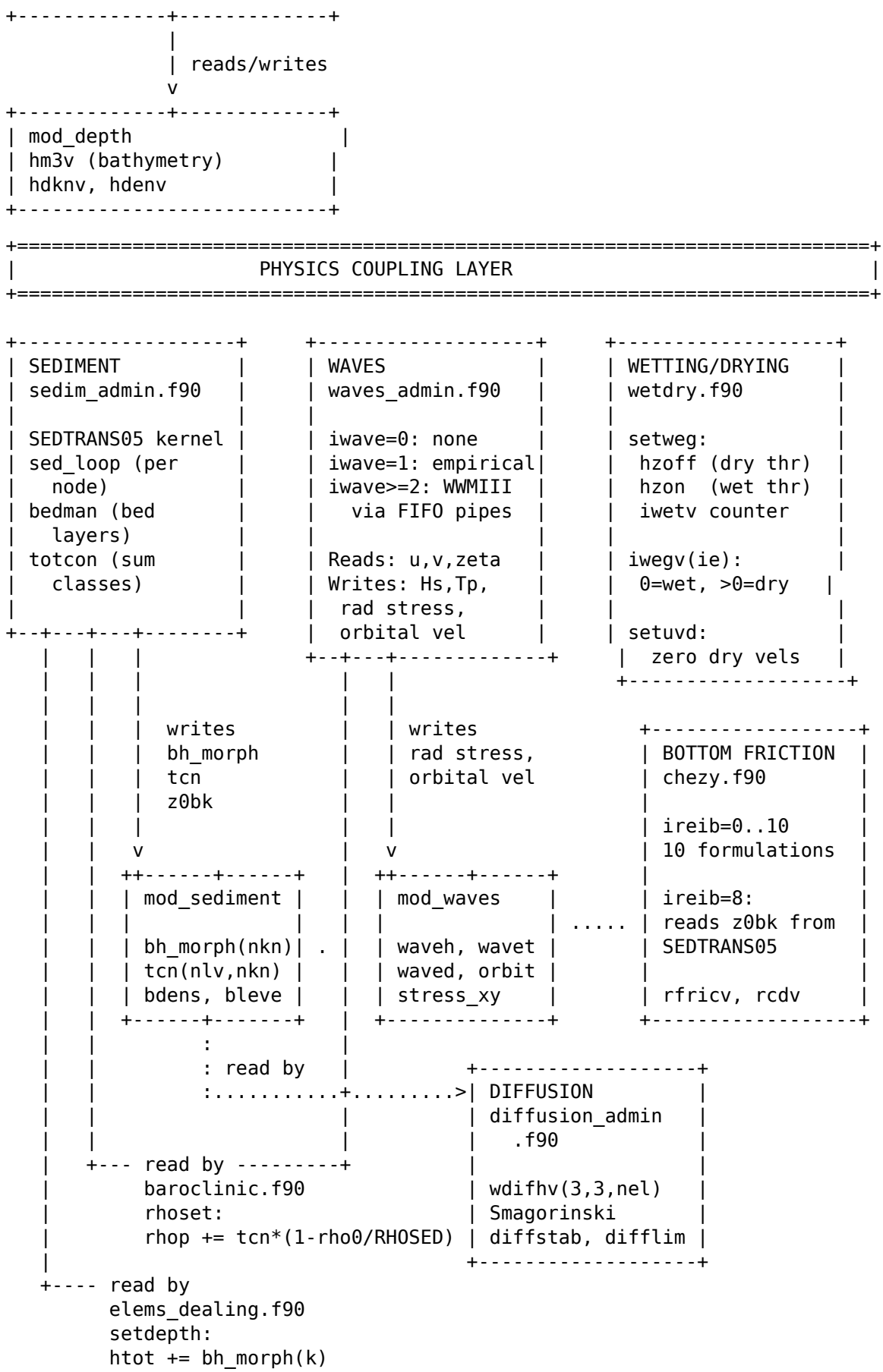
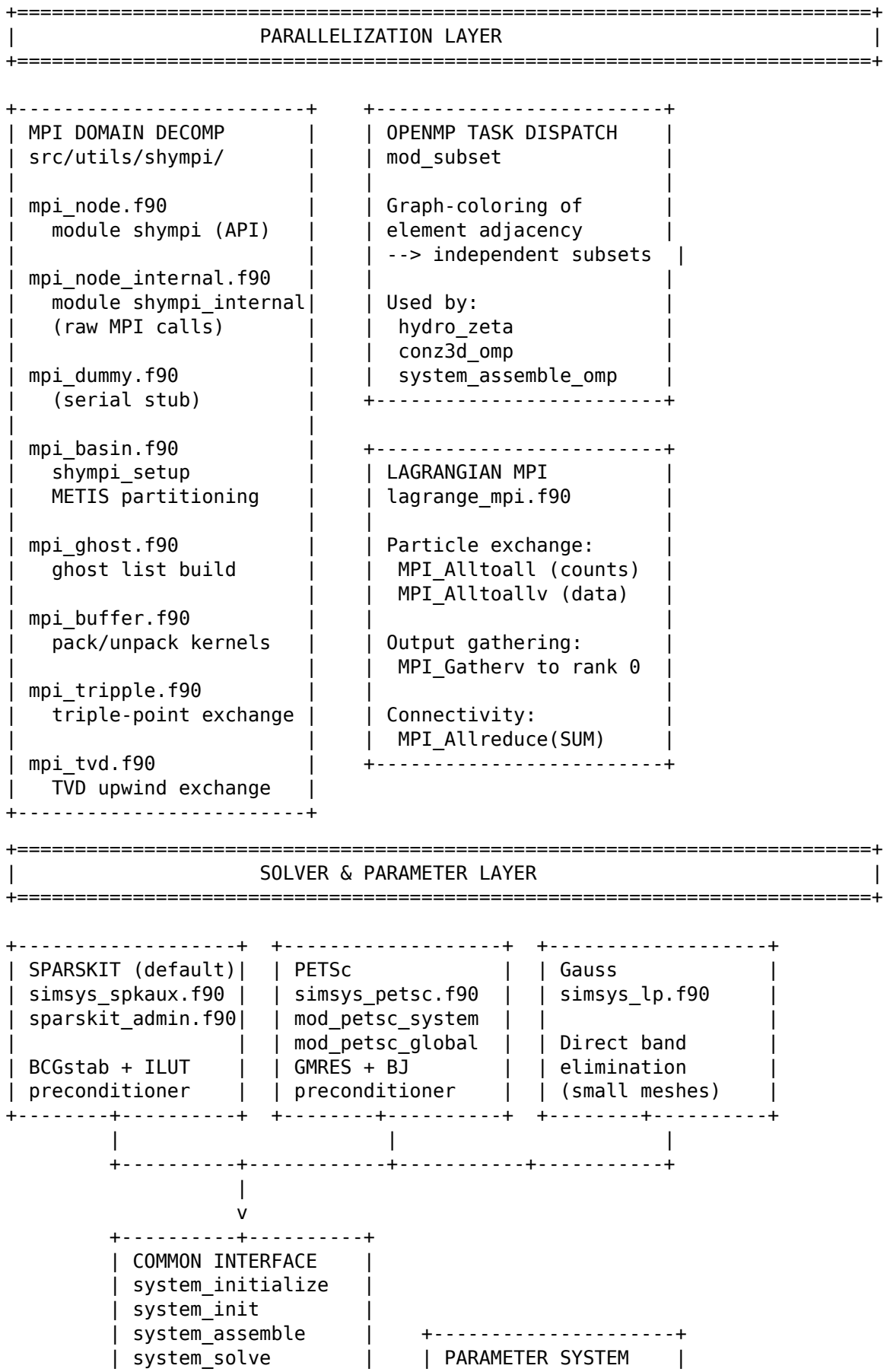


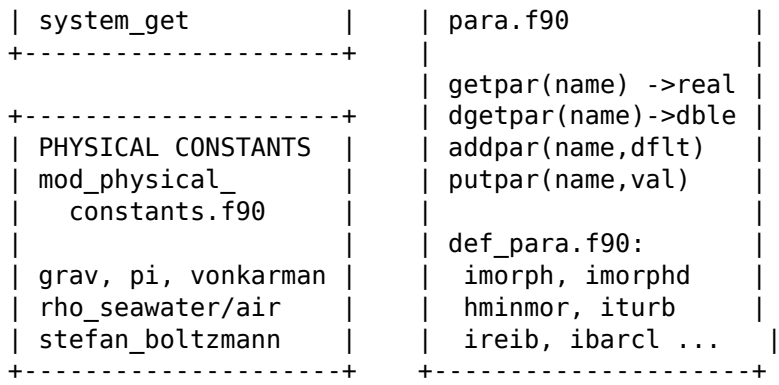
Figure 1: System Architecture Overview

```
                |                       |
   calls each   |                       | calls
   timestep     v                       v
+---------------+--------+   +---------+-----------+
|  HYDRODYNAMICS         |   |  SCALAR TRANSPORT   |
|  hydrodynamic.f90      |   |  run_scalar()       |
|                        |   |  [OMP TASK fan-out]  |
|  hydro_transports      |   +--+-------+-------+--+
|  hydro_zeta            |      |       |       |
|  hydro_transports_     |      v       v       v
|    final               |   barocl  tracer  bfm_run
|  hydro_vertical        |      |    compute
+-+---+---+---+---+-----+      |
  |   |   |   |   |            |
  |   |   |   |   |   +--------+----------+
  |   |   |   |   |   | BAROCLINIC        |
  |   |   |   |   |   | baroclinic.f90    |
  |   |   |   |   |   |                   |
  |   |   |   |   |   | T/S advection     |
  |   |   |   |   |   |   via conz3d_omp  |
  |   |   |   |   |   | rhoset (density)  |
  |   |   |   |   |   +--+-------+--------+
  |   |   |   |   |      |       |
  |   |   |   |   |      |reads  |writes
  |   |   |   |   |      |       v
  |   |   |   |   |      |  +----+----------+
  |   |   |   |   |      |  | mod_ts        |
  |   |   |   |   |      |  | tempv, saltv  |
  |   |   |   |   |      |  | rhov          |
  |   |   |   |   |      |  +---------------+
  |   |   |   |   |      |
  v   |   |   |   |      v
```


EXPLICIT
explicit
.f90
fxv, fyv
[momentum
forcing]
reads
mod_hydro
znv, unv
vnv, wlnv
utlnv,vtlnv
CONCENTRATION
concentration.f90
concentration_
omp.f90
conz3d_omp:
element pass
(FEM assembly)
node pass
(tridiag solve)
reads/writes
mod_diff_visc_fric
visv (viscosity)
difv (diffusivity)
difhv (horiz diffusion)
reads
writes
TURBULENCE
turbulence.f90
iturb=1: GOTM
iturb=2: k-eps
iturb=3: Munk-Anders
calls (iturb=1)
GOTM INTERFACE
fem_gotm_interface
.F90
do_gotm_turb()
DEPTH / GEOMETRY
elems_dealing.f90
setdepth:
hdkn (node depths)
hden (elem depths)
+-- imorph>0:
htot += bh_morph(k)
<--- reads bh_morph
from mod_sediment
+-- hminmor clamp
ele2node, set_area


reads/writes
mod_depth
hm3v (bathymetry)
hdknv, hdenv
PHYSICS COUPLING LAYER
SEDIMENT
sedim_admin.f90
SEDTRANS05 kernel
sed_loop (per node)
bedman (bed layers)
totcon (sum classes)
WAVES
waves_admin.f90
iwave=0: none
iwave=1: empirical
iwave>=2: WWMIII
via FIFO pipes
Reads: u,v,zeta
Writes: Hs,Tp,
rad stress,
orbital vel
WETTING/DRYING
wetdry.f90
setweg:
hzoff (dry thr)
hzon (wet thr)
iwetv counter
iwegv(ie):
0=wet, >0=dry
setuvd:
zero dry vels
writes
bh_morph
tcn
z0bk
writes
rad stress,
orbital vel
BOTTOM FRICTION
chezy.f90
ireib=0..10
10 formulations
ireib=8:
reads z0bk from
SEDTRANS05
rfricv, rcdv
mod_sediment
bh_morph(nkn)
tcn(nlv,nkn)
bdens, bleve
mod_waves
waveh, wavet
waved, orbit
stress_xy
read by
DIFFUSION
diffusion_admin .f90
wdifhv(3,3,nel)
Smagorinski
diffstab, difflim
read by
baroclinic.f90
rhoset:
rhop += tcn*(1-rho0/RHOSED)
read by
elems_dealing.f90
setdepth:
htot += bh_morph(k)


PARALLELIZATION LAYER
MPI DOMAIN DECOMP
src/utils/shympi/
mpi_node.f90
module shympi (API)
mpi_node_internal.f90
module shympi_internal
(raw MPI calls)
mpi_dummy.f90
(serial stub)
mpi_basin.f90
shympi_setup
METIS partitioning
mpi_ghost.f90
ghost list build
mpi_buffer.f90
pack/unpack kernels
mpi_tripple.f90
triple-point exchange
mpi_tvd.f90
TVD upwind exchange
OPENMP TASK DISPATCH
mod_subset
Graph-coloring of
element adjacency
--> independent subsets
Used by:
hydro_zeta
conz3d_omp
system_assemble_omp
LAGRANGIAN MPI
lagrange_mpi.f90
Particle exchange:
MPI_Alltoall (counts)
MPI_Alltoallv (data)
Output gathering:
MPI_Gatherv to rank 0
Connectivity:
MPI_Allreduce(SUM)
SOLVER & PARAMETER LAYER
SPARSKIT (default)
simsys_spkaux.f90
sparskit_admin.f90
BCGstab + ILUT
preconditioner
PETSc
simsys_petsc.f90
mod_petsc_system
mod_petsc_global
GMRES + BJ
preconditioner
Gauss
simsys_lp.f90
Direct band
elimination
(small meshes)
COMMON INTERFACE
system_initialize
system_init
system_assemble
system_solve
PARAMETER SYSTEM



Data Flow Summary

The following table summarizes all cross-module data flows through shared Fortran modules:

Shared Module	Key Arrays	Writers	Readers
mod_hydro	znv, unv, vnv, wlnv, utlnv, vtlnv	hydrodynamic.f90	explicit, concentration, baroclinic, turbulence, lagrange, sedim_admin
mod_ts	tempv, saltv, rhov	baroclinic.f90	turbulence (N-squared), rhoset (density), output
mod_diff_visc_fric	visv, difv, difhv	turbulence.f90, diffusion_admin.f90	hydrodynamic (vertical viscosity), concentration (diffusion)
mod_depth	hm3v, hdknv, hdenv	elems_dealing.f90	All physics modules (layer thickness)
mod_sediment	bh_morph, tcn, bdens	sedim_admin.f90	elems_dealing (depth), baroclinic (density)
mod_internal	fxv, fyv	explicit.f90	hydrodynamic (momentum RHS)
mod_geom_dynamic	iwegv, iwetv	wetdry.f90	All physics (skip dry elements)
mod_waves	waveh, orbit, stress	waves_admin.f90	sedim_admin (bed stress), explicit (radiation stress)
mod_roughness	z0bk	sedim_admin.f90	chezy.f90 (ireib=8)
mod_subset	indipendent_subset	Initialization	hydro_zeta, conz3d_omp (OMP tasks)
mod_lagrange	lgr_ar, nbdy	lagrange_main.f90, lagrange_mpi.f90	lagrange_inout (I/O), tracking loop

Single Timestep Execution Order

```
shyfem_run loop iteration:
  +-- set_dry / copy_uvz / copy_depth          [state bookkeeping]
  +-- sp111(2)                                  [apply boundary conditions]
  |
  +-- hydro                                     [HYDRODYNAMICS]
```

```
|   +-- set_diffusivity                        [update horizontal viscosity]
|   +-- [domain convergence loop]
|   |   +-- compute_dry_elements
|   |   +-- hydro_transports                   [compute u-hat, v-hat]
|   |   |   +-- bottom_friction
|   |   |   +-- set_explicit                   [explicit.f90]
|   |   |   |   +-- set_diff_horizontal        [horizontal momentum diffusion]
|   |   |   |   +-- set_advective              [nonlinear advection]
|   |   |   |   +-- set_barocl_new_interface   [baroclinic pressure forcing]
|   |   |   +-- [OMP PARALLEL DO] hydro_intern(ie,...)  [per-element vertical solve]
|   |   |       +-- penta_fact / penta_solve   [pentadiagonal Thomas algorithm]
|   |   +-- system_init / hydro_zeta(rqv)    [assemble free-surface FEM matrix]
|   |   |   +-- [OMP TASK per subset] system_assemble_omp
|   |   +-- system_solve(nkn,znv)            [SPARSKIT or PETSc solve for zeta]
|   |   +-- setweg(1,iw)                     [wet/dry check, iterate until stable]
|   +-- hydro_transports_final               [correct transports with new zeta]
|   +-- make_new_depth                        [setdepth with new zeta]
|   +-- hydro_vertical(dzeta)                [compute w from continuity]
|
+-- run_scalar                                [SCALAR TRANSPORT -- parallel tasks]
|   +-- [OMP TASK] barocl(1)                 [baroclinic.f90]
|   |   +-- [OMP TASK] scal_adv_nudge(temp)  [T advection+diffusion]
|   |   +-- [OMP TASK] scal_adv_nudge(salt)  [S advection+diffusion]
|   |   +-- [tasks sync] compute_heat_flux
|   |   +-- rhoset_shell --> [OMP PARALLEL DO] rhoset  [density from EOS]
|   +-- [OMP TASK] tracer_compute            [additional tracers]
|   +-- [OMP TASK] bfm_run                   [biogeochemical model]
|
+-- turb_closure                              [TURBULENCE]
|   +-- setm2n2                               [compute N-squared, M-squared]
|   +-- gotm_shell / munk_anderson_shell / keps_shell
|
+-- parwaves / sedi / submud                  [WAVES + SEDIMENT]
+-- bstress                                   [bottom shear stress]
+-- lagrange                                  [LAGRANGIAN PARTICLES]
+-- do_after                                  [output writing]
```

---

2. Core Hydrodynamics

`shyfem_subs.f90` – Main Simulation Loop (~2,100 lines)

Subroutine	Role
`shyfem_main`	Entry point: initialize -> run -> finalize
`shyfem_initialize(mpi_init)`	Full setup: STR file, MPI/OMP partitions, allocations, BCs
`shyfem_run(dtstep)`	Time-stepping loop
`shyfem_finalize`	Timing statistics, `system_finalize`
`run_scalar()`	OpenMP task fan-out for baroclinic, tracer, BFM

`hydrodynamic.f90` – Hydrodynamic Solver (~2,200 lines)

Solves the 3D shallow-water equations using a semi-implicit finite element scheme with weighting parameters: - `azpar`: time-weighting for free-surface in continuity - `ampar`: time-weighting for pressure gradient in momentum - `afpar`: Coriolis weighting - `atpar`: vertical viscosity weighting

Subroutine	Lines	Role
`hydro`	~210	Administrator: calls all sub-steps
`hydro_transports`	~125	Intermediate layer transports (u-hat, v-hat)
`hydro_intern(ie,...)`	~515	Per-element vertical pentadiagonal solve
`hydro_zeta(vqv)`	~400	FEM assembly for free-surface equation
`hydro_transports_final`	~120	Corrects transports using new zeta gradient
`hydro_vertical(dzeta)`	~205	Vertical velocity from continuity

Algorithm:

1. **`hydro_transports`** (`OMP PARALLEL DO` over elements): For each element, `hydro_intern` builds and solves a vertical pentadiagonal momentum system coupling u/v via Coriolis. Includes: vertical viscosity, bottom friction, wind stress, barotropic/baroclinic pressure, advection, wave radiation stress.
2. **`hydro_zeta`** (OMP TASK per subset): Assembles the global N x N sparse system `A*zeta = b` using lumped FEM mass matrix. Element 3x3 matrices computed from precomputed depth-integrated operators, accumulated via `system_assemble_omp` using graph-coloring subsets to avoid write conflicts.
3. **`system_solve`**: Inverts the assembled system (SPARSKIT iterative or PETSc).
4. **`hydro_transports_final`**: Corrects intermediate transports with actual zeta gradient.
5. **`hydro_vertical`**: Bottom-up continuity integration to recover vertical velocity `wlnv`.

Parallelization: `OMP PARALLEL DO` on element loops; `OMP TASK` with subset decomposition for FEM assembly; `OMP SIMD` on vertical layer loops. MPI ghost exchange after solve.

`explicit.f90` – Explicit Forcing Terms (~1,322 lines)

Computes all explicit (known at timestep start) forcing for the momentum equation RHS (`fxv`, `fyv`):

Subroutine	Role
`set_explicit`	Dispatcher: zeros fxv/fyv, calls each term
`set_diff_horizontal`	Horizontal momentum diffusion (Laplacian)
`set_advective`	Nonlinear advection u*grad(u)
`set_barocl_new_interface`	Baroclinic pressure gradient
`momentum_viscous_stability`	Fourier number stability check
`momentum_advective_stability`	Courant number stability check

Horizontal diffusion uses element-edge flux formulation; advection uses flux-weighted upwinding.

---

3. Scalar Transport

`concentration.f90` + `concentration_omp.f90` (~3,070 lines combined)

Advection-diffusion solver for any scalar field (T, S, tracers, sediment). Uses fractional-step approach with internal sub-cycling.

Key routines:

Subroutine	File	Role
`scal_adv` / `scal_adv_nudge` / `scal_adv_fact`	concentration.f90	Entry points for scalar advection
`scal3sh`	concentration.f90	Shell: BCs, sub-cycling, calls `conz3d_omp`
`conz3d_omp`	concentration_omp.f90	OMP-parallel 3D scalar solver
`conz3d_element(ie,...)`	concentration_omp.f90	Per-element horizontal FEM flux contributions
`conz3d_nodes(k,...)`	concentration_omp.f90	Per-node vertical tridiagonal solve

Algorithm: 1. **Element pass**: FEM basis functions compute horizontal advective/diffusive flux contributions to nodal tridiagonal arrays (`cdiag`, `clow`, `chigh`). Uses subset-based OMP tasks. 2. **MPI exchange**: `shympi_exchange_and_sum_3d_nodes` collects contributions across ranks. 3. **Node pass**: 1D vertical tridiagonal solve (Thomas algorithm) per node including vertical diffusion, advection, sinking, nudging, sources/sinks. 4. **Sub-cycling**: If Courant criterion requires, full timestep subdivided into `istot` sub-steps.

Performance optimization: Five large arrays (`cn`, `co`, `cdiag`, `clow`, `chigh`) are `allocatable, save` – cached between calls to avoid ~10 MB allocation/deallocation per timestep.

---

4. Turbulence Closure

`turbulence.f90` (~1,216 lines)

Computes turbulent viscosity `visv(l,k)` and diffusivity `difv(l,k)` at vertical layer interfaces.

`iturb`	Model	Parallelizable
1	GOTM (General Ocean Turbulence Model)	No (module-level scratch arrays)
2	Internal k-epsilon	Partial
3	Munk-Anderson (Ri-based)	Yes (`OMP PARALLEL DO`)

GOTM integration (`gotm_shell`): Calls `do_gotm_turb` per node serially. GOTM's module-global scratch arrays (`num`, `nuh`, `tke`, `eps`, `L`) make the node loop **not thread-parallelizable** without restructuring GOTM itself. This is the single largest OpenMP scalability bottleneck.

Munk-Anderson (`munk_anderson_shell`): Simple Ri-number scheme. `OMP PARALLEL DO SCHEDULE(DYNAMIC)` over nodes – fully parallelizable.

GOTM state persistence (`gotm_aux.f90`): Per-node-column arrays (`numv_gotm`, `nuhv_gotm`, `tken_gotm`, `eps_gotm`, `rls_gotm`) in `mod_gotm_aux`. Restart save/load via SHYFEM restart infrastructure.

---

5. Baroclinic (Density) Computations

`baroclinic.f90` (~1,072 lines)

Administers T/S advection-diffusion, computes seawater density from equation of state, maintains baroclinic pressure field.

Subroutine	Role
`barocl(mode)`	Administrator: mode=0 init, mode>0 advect T/S + recompute rho
`rhoset_shell(iterwant)`	Iterative density solver (residual < 1e-7)
`rhoset(resid)`	Per-node density: hydrostatic pressure integration + `sigma_stp` EOS

Algorithm (`rhoset`): Iterates because density depends on pressure and pressure depends on density. For each node (OMP PARALLEL DO): 1. Integrate pressure downward layer by layer 2. Call `sigma_stp(salt, temp, pres)` – UNESCO equation of state 3. Store `rhov(l,k) = sigma - sigma0`

Sediment density coupling (when `imorphd > 0`):

```
rhop = rhop + tcn(l,k) * (1. - rho0/real(RHOSED))
```

With RHOSED=2650 kg/m3, 1 kg/m3 of suspended sediment increases effective density by ~0.61 kg/m3.

---

6. Element and Depth Operations

`elems_dealing.f90` (~1,083 lines)

Geometric infrastructure: element-to-node data scatters, depth/layer thickness computation, area computation.

`setdepth` (~278 lines) – the most complex routine. Handles three coordinate systems: 1. **Sigma layers** (l <= nsigma): `h = -hsig * hldv(l)` 2. **Adaptive z-surface layers**: `compute_zadaptive_info` with `cadapt` coefficients 3. **Pure z-layers** (l > sigma+adaptive): fixed thicknesses from `hlv`

Morphological feedback (when `imorph > 0`):

```
htot = max(htot + bh_morph(k), hminmor)
```

Directly modifies water column thickness seen by momentum equations each timestep.

Parallelization: Element scatter-add loop is serial (race condition on shared node arrays). Node normalization loop uses `OMP PARALLEL DO SCHEDULE(STATIC)`.

---

7. Physics and Coupling Modules

7.1 Parameter System (`src/utils/generic/para.f90`)

Global runtime parameter registry. Every tunable constant and physics switch is stored/retrieved through `module para`.

Function	Description
`getpar(name)`	Returns scalar `real` value (linear scan)
`dgetpar(name)`	Returns `double precision`
`addpar(name, default)`	Registers parameter with default (startup only)
`putpar(name, value)`	Overwrites existing value

Flow: Init routines call `addpar` to register defaults -> STR file parsing overrides via `putpar` -> Runtime access via `getpar` (cached with `icall=0` pattern in hot paths).

7.2 Sediment Transport (`sedim_admin.f90`)

SEDTRANS05-based orchestrator for cohesive (mud) and non-cohesive (sand) transport.

Execution flow per timestep:

```
sedi()
+-- tvd_grad_2d          [bed slope gradients]
+-- sed_loop             [SEDTRANS05 per node: erosion/deposition]
+-- bedload              [integrate bedload fluxes]
+-- scn_compute          [advect/diffuse suspended concentrations]
+-- bedman               [update bed layers, compute bdh]
|   +-- bh_morph(k) += bdh(k)   [accumulate morphological offset]
+-- totcon               [sum all classes into tcn(nlv,nkn)]
```

7.3 Two-Way Sediment-Hydrodynamics Coupling

Three feedback channels:

Channel	Write Side	Read Side	Control
Bathymetric	`bh_morph(k) += bdh(k)` in `bedman`	`htot += bh_morph(k)` in `setdepth`	`imorph > 0`
SSC density	`tcn(l,k)` aggregated in `totcon`	`rhop += tcn*(1-rho0/RHOSED)` in `rhoset`	`imorphd > 0`
Roughness	`z0bk(k)` from SEDTRANS05	`cdf(h,z0)` in `bottom_friction`	`ireib = 8`

Two-Way Coupling Diagram

7.4 Coupling Architecture Diagram

```
Hydrodynamics (momentum/continuity)
    ^ depth modified by bh_morph           <-- setdepth (elems_dealing.f90)
    ^ baroclinic pressure modified by tcn  <-- rhoset (baroclinic.f90)
    ^ bottom friction via z0bk             <-- bottom_friction (chezy.f90)
    ^ wave radiation stresses              <-- read_wwm (waves_admin.f90)
    ^ eddy viscosity from GOTM             <-- turb_closure
    |
    v velocities, water levels
    v surface velocities --> WWM            --> write_wwm
    |
Sediment Transport (sedi)
    v bedload + suspended flux --> bedman
    v bh_morph accumulation [m]           --> feeds back to depth
    v tcn concentration [kg/m3]           --> feeds back to density
    v z0bk roughness length [m]           --> feeds back to friction
    |
Wave Model (WWM or parwaves)
    v orbital velocity, bottom stress     --> SEDTRANS05 input
    v radiation stresses                  --> momentum forcing
    v Stokes drift                        --> concentration transport
```

7.5 Waves (`waves_admin.f90`)

Three modes: `iwave=0` (none), `iwave=1` (empirical parametric), `iwave>=2` (WWMIII coupled via named FIFO pipes). WWMIII coupling exchanges ~15 variables bidirectionally every `dtwave` seconds.

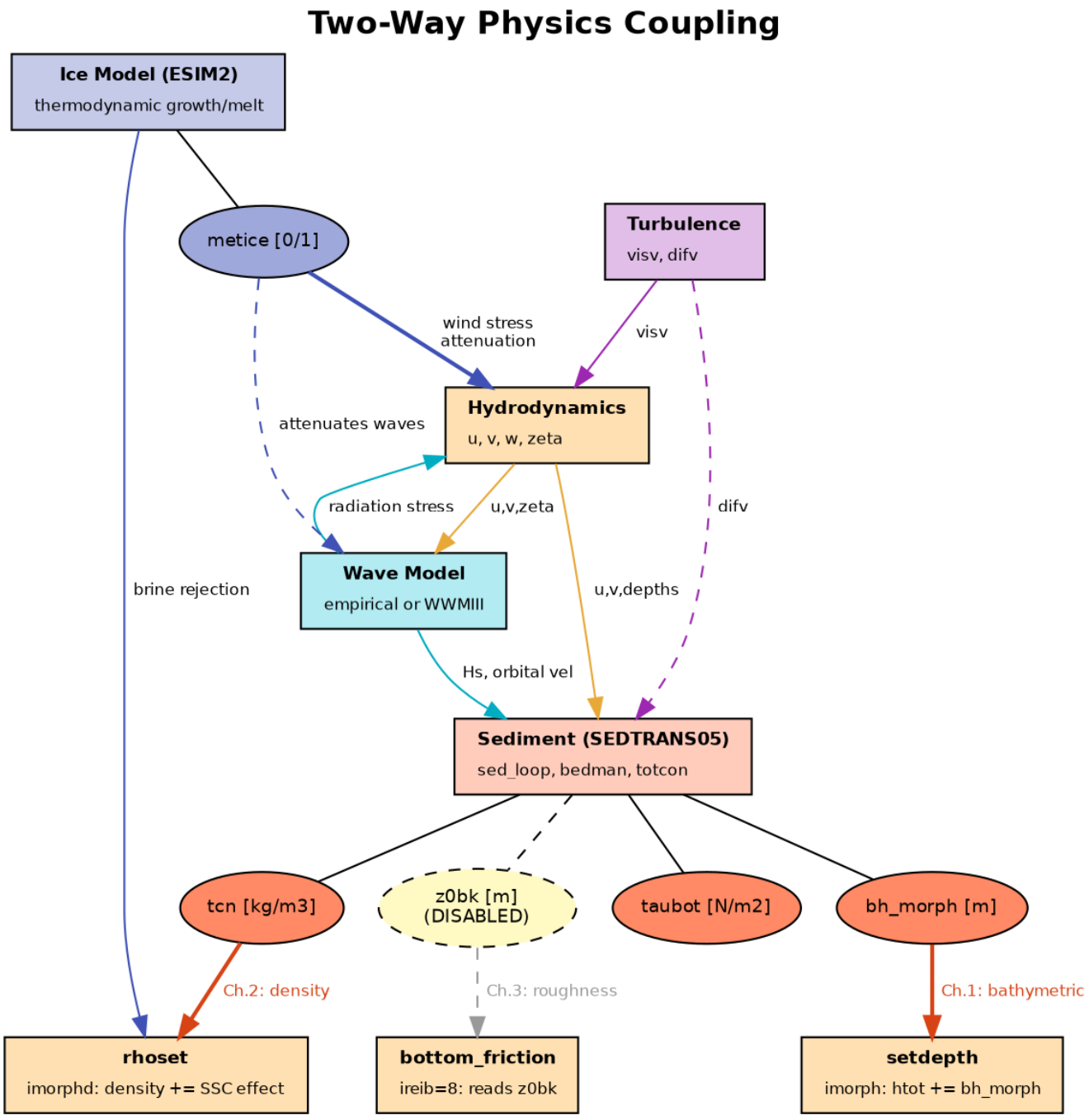


Figure 2: Two-Way Physics Coupling

7.6 Wetting and Drying (`wetdry.f90`)

Threshold-based element activation/deactivation using `iwegv(ie)` (0=wet, >0=dry node count). Parameters: `hzoff` (drying threshold), `hzon` (re-wetting threshold), `iwait=5` (oscillation guard).

7.7 Bottom Friction (`chezy.f90`)

Ten formulations selected by `ireib` (0-10): constant, Strickler, Chezy, drag coefficient, log-law roughness, sediment-feedback, fluid mud, hybrid. Area-code indexed for spatially varying friction. OMP-parallelized element loop.

7.8 Diffusion (`diffusion_admin.f90`)

Computes diffusion weight matrix `wdifhv(3,3,nel)` from FEM shape function gradients. Three algorithms (`idtype=0,1,2`) with monotonicity enforcement. Smagorinski sub-grid closure available.

7.9 Physical Constants (`mod_physical_constants.f90`)

All constants as Fortran `parameter` (compile-time, zero runtime overhead):

Constant	Value
`grav_dp/sp`	9.81 m/s2
`pi_dp/sp`	3.14159265358979...
`vonkarman`	0.4
`rho_seawater`	1025.0 kg/m3
`stefan_boltzmann`	5.670374419e-8 W/m2/K4

---

8. MPI Parallelization Infrastructure

MPI and OpenMP Architecture Diagram

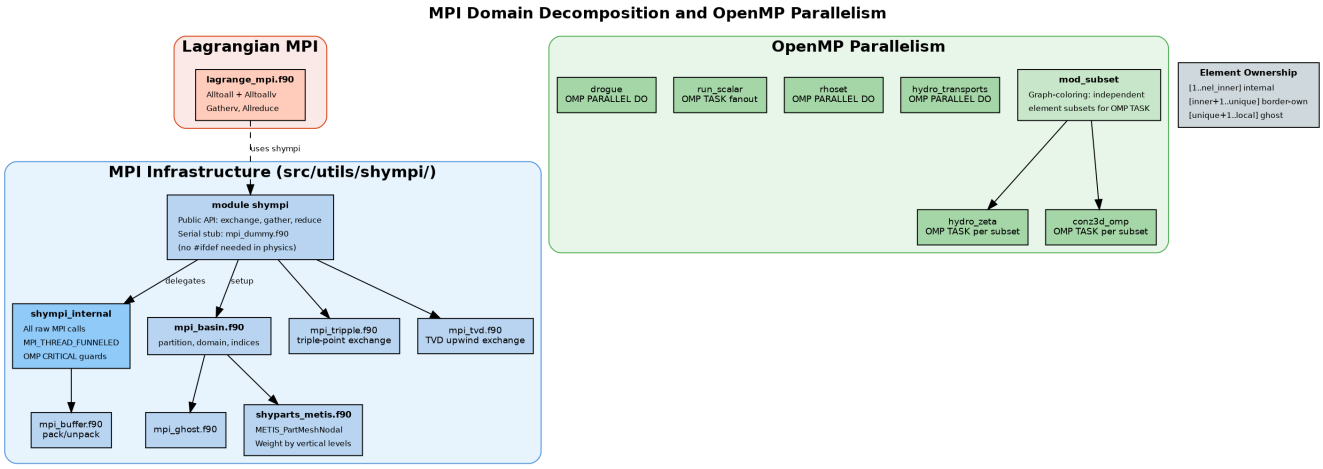


Figure 3: MPI Domain Decomposition and OpenMP Parallelism

8.1 Architecture

Node-based domain decomposition via METIS. Each rank holds inner nodes/elements plus a ghost layer. The `module shympi` (`mpi_node.f90`) is the single public facade; actual MPI calls are isolated

in `module shympi_internal` (`mpi_node_internal.f90`). Serial builds use `mpi_dummy.f90` – identical interface, no MPI calls, no `#ifdef` guards needed in physics code.

8.2 Key Data Structures

```
nkn_global > nkn_local > nkn_unique == nkn_inner
nel_global > nel_local >= nel_unique >= nel_inner
```

Variable	Meaning
`my_id`	Rank (0-based)
`n_threads`	Total MPI ranks
`id_node(nkn)`	Domain owner of each local node
`id_elem(0:3,nel)`	Domain ownership: `(0)=count, (1)=primary, (2-3)=secondary`
`ghost_nodes_in/out`	Inner/outer ghost node lists per neighbor
`ip_int_node/elem`	Local-to-global index mapping

8.3 Element Ownership Encoding (`id_elem`)

Pattern	Meaning
`(1, my_id, -1, -1)`	Fully internal element
`(2, my_id, id, -1)`	Border: 2 own nodes, 1 from `id`
`(2, id, my_id, -1)`	Border: 1 own node, primary owner is `id`
`(3, id1, id2, id3)`	Triple-point: 3 different domains

8.4 Ghost Exchange Kernel (`mpi_node_internal.f90`)

For each neighbor rank: 1. Post `MPI_Irecv` for outer ghost buffer 2. Pack inner ghost nodes via `to_buffer_r/i` (`mpi_buffer.f90`) 3. Post `MPI_Isend` 4. `MPI_WaitAll` 5. Unpack received data via `from_buffer_r/i`

All MPI calls wrapped in `!$OMP CRITICAL` for hybrid safety. Thread level: `MPI_THREAD_FUNNELED`.

8.5 Domain Decomposition Setup

```
shympi_setup (mpi_basin.f90)
  +-- handle_partition --> do_partition (METIS)
  +-- make_domain      --> flag inner/border/ghost
  +-- make_index       --> sorted: inner first, ghost after
  +-- adjust_indices   --> compute nkn_inner/unique/local
  +-- transfer_domain  --> reindex basin to local numbering
  +-- ghost_handle     --> build ghost lists, verify
  +-- shympi_gather    --> domain sizes
```

METIS partitioning (`shyparts_metis.f90`): Weighted by `vwgt(k) = max(1, ilhkv(k))` – nodes with more vertical levels get higher weight for load balancing.

8.6 Triple-Point Exchange (`mpi_tripple.f90`)

Elements whose 3 nodes belong to 3 different domains require special handling beyond standard two-way ghost exchange. Point-to-point `MPI_Isend/Irecv/WaitAll` per triple-point carrying `(lmax, area, u(1:lmax), v(1:lmax))`.

8.7 TVD Scalar Advection MPI (`mpi_tvd.f90`)

Dedicated infrastructure for Total Variation Diminishing transport: upwind concentration values from neighboring elements on other ranks. Initialized once, reused per timestep. Adds a third communication pattern.

8.8 MPI File Summary

File	Role
`mpi_node.f90`	`module shympi` – public API, all interface blocks
`mpi_dummy.f90`	Serial stub – identical interface
`mpi_node_internal.f90`	`module shympi_internal` – all raw MPI calls
`mpi_basin.f90`	`shympi_setup`, domain decomposition orchestration
`mpi_ghost.f90`	Ghost list construction and verification
`mpi_buffer.f90`	Pack/unpack kernels for ghost exchange
`mpi_tripple.f90`	Triple-point exchange protocol
`mpi_tvd.f90`	TVD scalar advection cross-rank lookup
`shyparts_metis.f90`	METIS weighted mesh partitioning
`shyparts_util.f90`	Partition quality analysis
`mpi_general.f90`	Sorted index for binary search
`mpi_debug.f90`	Per-rank debug output
`mpi_check.f90`	Runtime consistency checking
`mpi_time.f90`	Solver timing accumulators

---

9. Lagrangian Particle Tracking

Lagrangian System Diagram

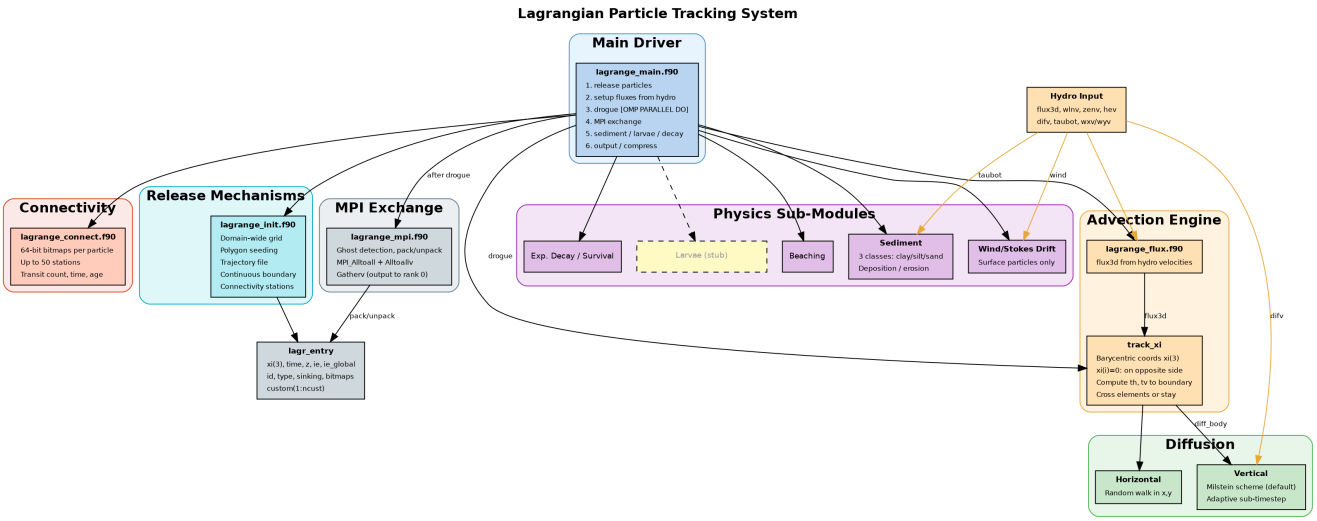


Figure 4: Lagrangian Particle Tracking System

9.1 Overview

The Lagrangian particle tracking system spans 19 source files in `src/shyfem/` implementing particle advection, diffusion, sediment interaction, connectivity analysis, and MPI inter-rank exchange. The

main driver (`lagrange_main.f90`) executes a 6-step pipeline each timestep: (1) release particles, (2) setup fluxes from hydrodynamics, (3) advect via `drogue` (OMP PARALLEL DO), (4) MPI exchange, (5) sediment/larvae/decay physics, (6) output/compress.

9.2 Advection Engine

Particle advection uses **barycentric coordinates** (`track_xi` in `lagrange_track.f90`). Each particle's position within a triangle is described by `xi(3)` where `xi(i)=0` means the particle is on the edge opposite node `i`. The advection algorithm:

1. Compute velocity at particle position from element flux field (`flux3d`)
2. Calculate time to reach each boundary (`th`, `tv` for horizontal/vertical)
3. If boundary reached within timestep: cross to neighbor element and continue
4. If not: update `xi(3)` in current element

This is exact for piecewise-linear velocity fields – no trajectory integration error within elements.

9.3 Diffusion

Vertical diffusion (`lagrange_vertical.f90`): Milstein scheme (default) – a stochastic differential equation integrator that corrects for spatially-varying diffusivity gradients. Uses adaptive sub-timestepping to prevent particles from overshooting layer boundaries.

Horizontal diffusion (`lagrange_horizontal.f90`): Random walk displacement scaled by `sqrt(2 * Kh * dt)` where `Kh` is the local horizontal diffusivity.

9.4 Sub-Modules

Module	File	Description
Sediment	`lagrange_sedim.f90`	3 classes (clay/silt/sand), deposition when tau < tau_crit, erosion
Wind/Stokes drift	`lagrange_stokes.f90`	Surface particles only, wind-proportional drift
Beaching	`lagrange_beach.f90`	Particle stranding on dry elements
Decay	`lagrange_decay.f90`	Exponential decay / survival probability
Larvae	`lagrange_larvae.f90`	Stub (not implemented)
Connectivity	`lagrange_connect.f90`	64-bit bitmaps per particle, up to 50 stations, transit count/time/age

9.5 Release Mechanisms (`lagrange_init.f90`)

Five release modes: domain-wide grid seeding, polygon-based seeding, trajectory file replay, continuous boundary release, and connectivity station release. In MPI mode, particles are only released on the rank owning their element (`id_elem(1,ie) == my_id`).

9.6 MPI Exchange (`lagrange_mpi.f90`, ~497 lines)

MPI support for particle migration between ranks. Added 2026.

Particle Data Structure (`lagrange_data.f90`)

```
type :: lagr_body
    double precision :: xi(3)     ! barycentric coordinates
    double precision :: time
    real :: z                     ! relative vertical [0-1]
    integer :: ie                 ! local element number
    integer :: ie_global          ! global element number (MPI key)
    integer :: l                  ! layer
end type

type :: lagr_entry
    integer(kind(bm_kind)) :: bitmap_in, bitmap_out  ! connectivity
    double precision :: sinking
    real, allocatable :: custom(:)
    integer :: id, type
    type(lagr_body) :: init, actual
end type
```

Serialization Format (13 + ncust doubles per particle)

Index	Field
1-3	`xi(1:3)` barycentric coords
4	`time`
5	`z`
6	`ie_global`
7	layer
8	id
9	type
10	sinking
11-12	bitmap_in, bitmap_out
13	ncust count
14..13+ncust	custom values

Exchange Protocol (`lgr_mpi_exchange`)

1. **Count**: `send_counts(irank)` per export
2. **`MPI_Alltoall`**: exchange counts
3. **Pack**: `lgr_mpi_pack_particle` into send buffer
4. **`MPI_Alltoallv`**: exchange particle data
5. **Unpack**: `lgr_mpi_unpack_particle` + `lgr_mpi_global_to_local_elem`
6. **Mark dead**: set `ie = 0` for exported particles

Other Key Routines

Routine	Role
`lgr_mpi_init`	Allocate export arrays, set pack size
`lgr_mpi_check_ghost(i, ie)`	Check `id_elem(1,ie) /= my_id`, mark for export
`lgr_mpi_mark_export(i, dest)`	Thread-safe (`OMP CRITICAL`) export accumulation
`lgr_mpi_gather_output(iu, dtime)`	`MPI_Gatherv` all particles to rank 0 for I/O
`lgr_mpi_reduce_connect`	`MPI_Allreduce(SUM)` connectivity matrices

Integration Points

- `lagrange_main.f90`: `lgr_mpi_init` at startup; `lgr_mpi_exchange` after `drogue(dtime)`; `lgr_mpi_gather_output` for I/O; `lgr_mpi_reduce_connect` before output
- `lagrange_init.f90`: Rank-aware particle release (`id_elem(1,ie) /= my_id` -> skip)
- `lagrange_inout.f90`: Sets `ie_global = ip_int_elem(ie)` on particle insertion and file input

10. Ecology and Biogeochemistry

Ecology Integration Diagram

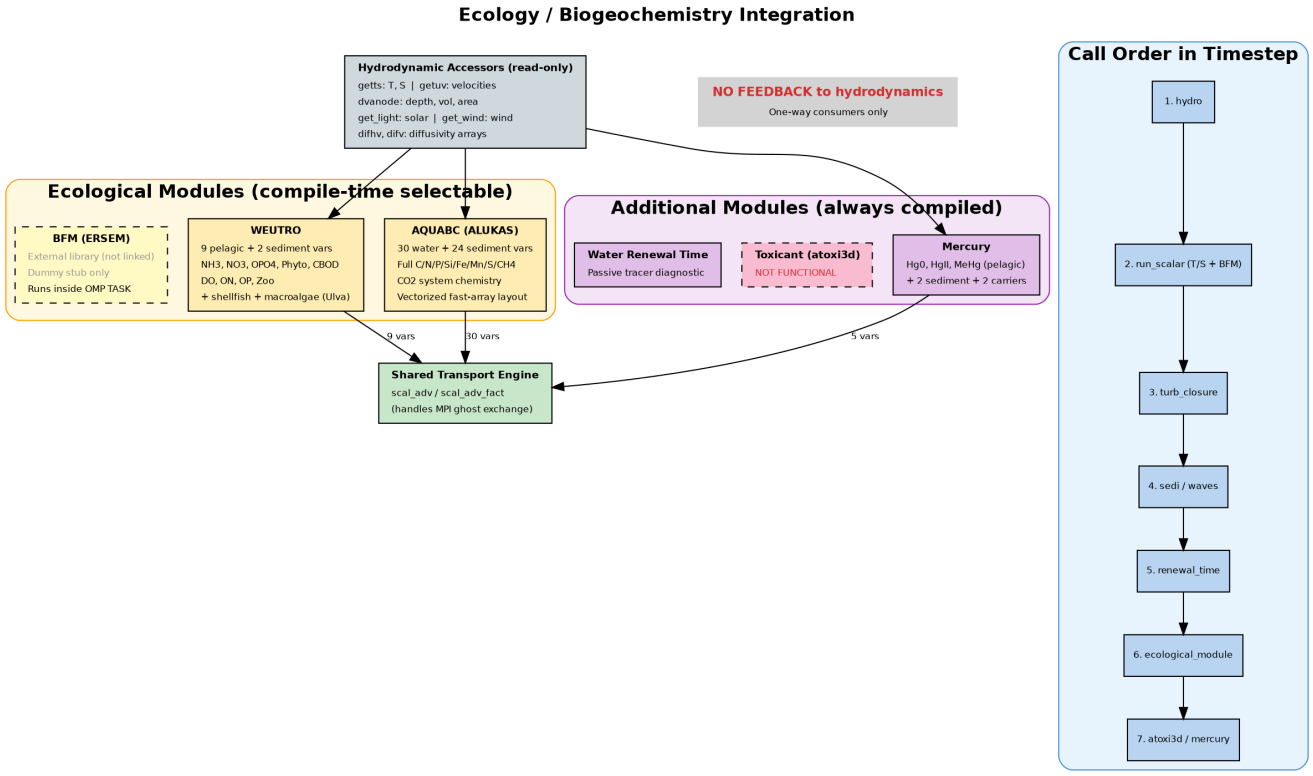


Figure 5: Ecology / Biogeochemistry Integration

10.1 Architecture

All ecological modules are **one-way consumers** of hydrodynamic fields – they read T, S, velocities, depth, light, and wind through accessor functions (`getts`, `getuv`, `dvanode`, `get_light`, `get_wind`) but produce **no feedback** to the hydrodynamic solver. Transport of ecological state variables uses the shared `scal_adv` / `scal_adv_fact` scalar advection engine, which handles MPI ghost exchange internally.

10.2 WEUTRO (Eutrophication Model)

Location: `src/contrib/ecological/weutro/` (~12 files)

9 pelagic state variables + 2 sediment state variables:

Variable	Description
NH3	Ammonia nitrogen
NO3	Nitrate nitrogen
OPO4	Orthophosphate
Phyto	Phytoplankton biomass
CBOD	Carbonaceous BOD
DO	Dissolved oxygen
ON	Organic nitrogen

Variable	Description
OP	Organic phosphorus
Zoo	Zooplankton
+2 sediment	Benthic demand layers

Extensions include shellfish filtration and macroalgae (Ulva) modules. Each variable is advected/diffused independently through `scal_adv_fact`. Reactions computed per node per timestep.

10.3 AQUABC (ALUKAS – Advanced Biogeochemistry)

Location: `src/contrib/ecological/aquabc/` (~25 files)

30 water column + 24 sediment state variables covering full C/N/P/Si/Fe/Mn/S/CH4 cycles with CO2 system chemistry. Uses a **vectorized fast-array layout** for computational efficiency.

Key features: - Full nitrogen cycle: NH4, NO3, nitrification, denitrification - Phosphorus: PO4, organic P, sediment P release - Carbon: DOC, POC, CO2 system (pH, alkalinity, DIC) - Metals: Fe/Mn redox cycling - Sediment diagenesis: 24-variable early diagenesis model

10.4 BFM (ERSEM)

Status: Stub only – external library not linked. Interface exists in `bfm_run` called as an OMP TASK inside `run_scalar`, but the actual BFM library is not compiled in.

10.5 Mercury Module

Location: `src/shyfem/mercury.f90` + supporting files

3 pelagic species (Hg0, HgII, MeHg) + 2 sediment + 2 carrier variables. Always compiled but only active when mercury parameters are set. Transported via `scal_adv`.

10.6 Toxicant Module (atoxi3d)

Status: NOT FUNCTIONAL. Code exists but is not operational.

10.7 Water Renewal Time

Passive tracer diagnostic: initializes concentration to 1.0 everywhere, lets it advect/diffuse with zero boundary conditions. The time for concentration to fall below a threshold gives the water renewal timescale.

10.8 Timestep Call Order

```
1. hydro                    [hydrodynamics]
2. run_scalar (T/S + BFM)  [scalar transport, OMP tasks]
3. turb_closure             [turbulence]
4. sedi / waves             [sediment, waves]
5. renewal_time             [water age diagnostic]
6. ecological_module        [WEUTRO or AQUABC]
7. atoxi3d / mercury        [contaminant transport]
```

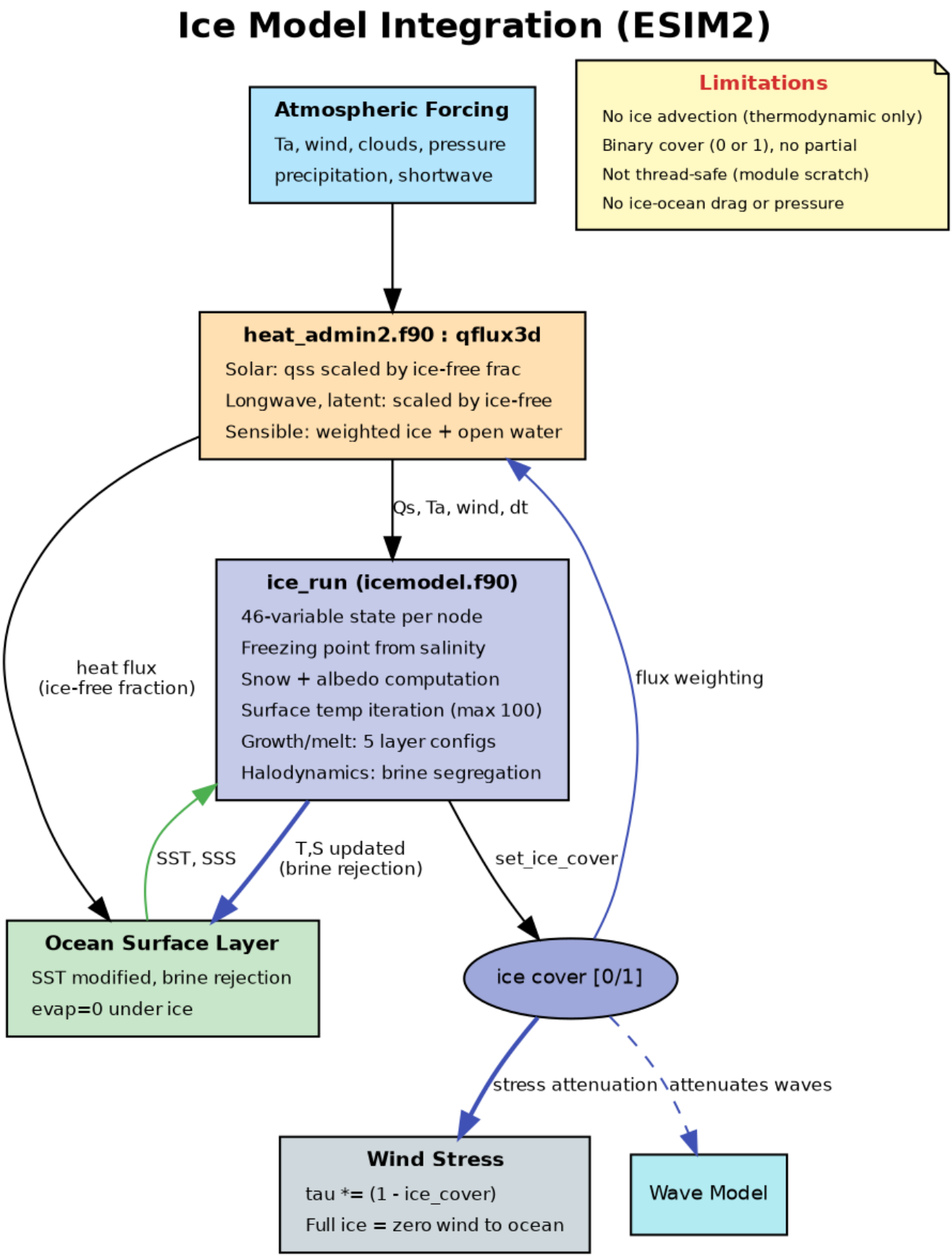


Figure 6: Ice Model Integration (ESIM2)

11. Ice Model

Ice Model Diagram

11.1 Overview

SHYFEM integrates the **ESIM2 thermodynamic ice model** (`icemodel.f90`). This is a thermodynamic-only model – it computes ice growth, melt, and snow accumulation at each node but does **not advect ice**. The model maintains a 46-variable state per node.

11.2 Physics

Freezing point: Computed from local salinity (depression formula).

Surface temperature iteration: Newton-Raphson iteration (max 100 steps) to balance radiative, sensible, and latent heat fluxes at the ice/snow surface.

Growth/melt configurations: 5 layer configurations depending on ice/snow state: 1. Open water – standard heat flux 2. Ice only, growing – conductive flux through ice 3. Ice only, melting – surface/basal melt 4. Ice + snow, growing – insulating snow layer 5. Ice + snow, melting – snow melt then ice melt

Halodynamics: Brine segregation – when ice forms, salt is rejected into the ocean surface layer, increasing SSS.

11.3 Coupling Channels

Channel	Direction	Mechanism
Heat flux weighting	Ice → Atmosphere	`qflux3d` scales solar/longwave/latent by ice-free fraction
Wind stress attenuation	Ice → Ocean	`tau *= (1 - ice_cover)` – full ice = zero wind to ocean
SST/SSS modification	Ice → Ocean	Brine rejection increases salinity; SST clamped at freezing point
Evaporation suppression	Ice → Ocean	`evap = 0` under ice cover
Wave attenuation	Ice → Waves	Dashed coupling (attenuates wave energy under ice)

11.4 Limitations

1. **No ice advection** – thermodynamic only, no ice drift or ridging
2. **Binary ice cover** – 0 (open) or 1 (fully covered), no partial cover fraction
3. **Not thread-safe** – module-level scratch arrays (same issue as GOTM)
4. **No ice-ocean drag** – no momentum transfer from ice to water
5. **No ice pressure** – no ice internal stress or rheology

11.5 Heat Flux Administration (`heat_admin2.f90`)

The `qflux3d` routine administers all surface heat fluxes with ice-aware weighting: - Solar (`qss`): scaled by ice-free fraction - Longwave, latent: scaled by ice-free fraction - Sensible heat: weighted combination of ice and open-water contributions

The ice model receives atmospheric forcing (air temperature, wind speed, cloud cover, pressure, precipitation, shortwave radiation) and returns the ice state which feeds back into the heat flux partitioning.

---

12. Sediment Transport (Detailed)

Sediment System Diagram

12.1 SEDTRANS05 Pipeline

The sediment system executes an 8-step pipeline each timestep:

Step	Routine	Description
1	`tvd_grad_2d`	Compute bed slope gradients for bedload direction
2	`sed_loop`	SEDTRANS05 physics kernel per node
3	Export `taubot`	Bottom shear stress to `mod_bstress`
4	`bedload`	Integrate bedload flux divergence
5	`scn_compute`	Advect/diffuse suspended sediment concentrations via `scal_adv`
6	`bedman`	Update bed layers, accumulate `bh_morph`
7	`smooth_node`	Smooth morphological changes
8	`totcon`	Sum all grain classes into `tcn(nlv,nkn)`

12.2 SEDTRANS05 Physics Kernel

The SEDTRANS05 kernel (called per node in `sed_loop`) executes a chain of physics computations:

1. **OSCIL**: Linear wave theory – wave orbital velocities at the bed
2. **THRESH**: Critical velocities for sediment motion initiation
3. **FRICFAC**: Grant-Madsen boundary layer – combined wave-current friction factor
4. **vmixcoef**: Vertical diffusion coefficient for suspended sediment
5. **activelayer**: Active layer mixing depth computation

12.3 Transport Branches

The kernel branches based on sediment type:

Cohesive (mud, grain size < 63 μm): - Erosion: `EROS1` per bed layer (depth-dependent erosion rate) - Deposition: Krone formula (`dep = ws * C * (1 - tau/tau_cd)` when `tau < tau_cd`) - Drag reduction coefficient for mud-laden flows

Non-cohesive (sand, grain size ≥ 63 μm): Five bedload/total-load formulas selected by `IOPT`: 1. Engelund-Hansen 2. Einstein 3. Bagnold 4. Yalin 5. **Van Rijn (default)** – separate bedload and suspended load

12.4 Grain Size Classes

`nscls` classes read from STR file. Classes 1 through `nbcc` are cohesive (grain size < 63 μm); classes `nbcc+1` through `nscls` are non-cohesive. Each class has its own suspended concentration field `scn(nlv,nkn,icls)` transported independently.

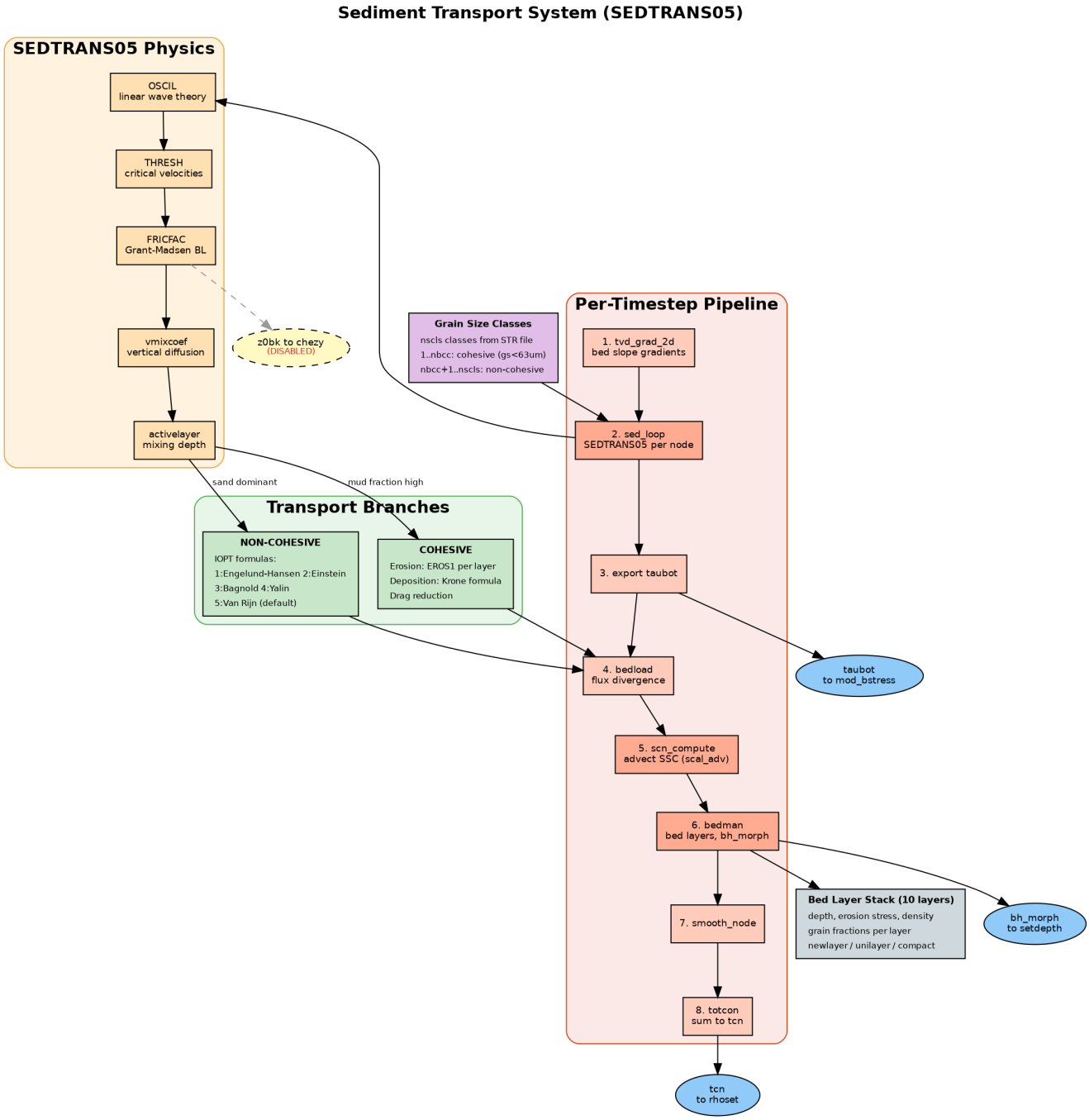


Figure 7: Sediment Transport System (SEDTRANS05)

12.5 Bed Layer Stack

10-layer bed structure per node, each layer storing: - Depth (thickness) - Erosion stress threshold - Bulk density - Grain size fractions per class

Layer management routines: - `newlayer`: Create new surface layer from deposition - `unilayer`: Merge thin layers - `compact`: Consolidation of cohesive layers

12.6 Morphological Acceleration (MORPHO)

When `imorph > 0`, morphological changes can be accelerated by a factor `morfac`:

```
bh_morph(k) = bh_morph(k) + bdh(k) * morfac
```

This allows faster-than-real-time evolution of bathymetry while maintaining hydrodynamic timestep constraints.

12.7 Roughness Feedback (DISABLED)

The SEDTRANS05 kernel computes a bed roughness `z0bk` from grain size and bedform geometry. This was intended to feed back to the Chezy bottom friction routine (`ireib=8`), but the feedback path is **currently disabled** (commented out in source). When active, it would provide dynamic roughness that changes as the bed evolves.

12.8 Two-Way Coupling Summary

Output	Target	Active
`bh_morph(k)`	`setdepth` – modifies water column depth	Yes (`imorph > 0`)
`tcn(nlv,nkn)`	`rhoset` – modifies density via suspended sediment	Yes (`imorphd > 0`)
`taubot`	`mod_bstress` – bottom shear stress diagnostic	Yes
`z0bk`	`chezy.f90` bottom friction (`ireib=8`)	**DISABLED**

---

13. Build System

Hierarchy

- `Makefile` (top-level): User-facing targets (`make fem`, `make petsc`, `make clean`)
- `Rules.make`: Central configuration – compiler flags, profiles, option validation
- `src/shyfem/Makefile`: Object lists, solver-conditional compilation, link step
- 68 sub-Makefiles with `.PHONY` declarations

Compiler Profiles

Profile	Optimization	Debug	FPE Traps	Use Case
`NORMAL` (default)	`-O2`	Yes	Yes	Development
`CHECK`	None	Yes	Yes + bounds	Debugging

Profile	Optimization	Debug	FPE Traps	Use Case
SPEED	-O3 -march=native -funroll-loops -ftree-vectorize	No	No	Production
SPEED_PGO_GEN	SPEED + -fprofile-generate	No	No	PGO training
SPEED_PGO_USE	SPEED + -fprofile-use	No	No	PGO production

Note: In NORMAL, -ffpe-trap suppresses !$OMP SIMD directives at runtime. SIMD hints are only effective in SPEED profiles.

Special Compilation Flags

FFLAG_SPECIAL includes -fallow-argument-mismatch (gfortran) for legacy MPI Fortran interfaces. Applied to: - mpi_node_internal.f90 - lagrange_mpi.f90

Key Build Commands

```
# Serial build
make fem

# MPI build (OpenMPI, not Intel MPI)
PATH=/usr/bin:$PATH make fem OMP=false MPI=true PARTS=METIS METISDIR=/usr

# Hybrid MPI+OpenMP
PATH=/usr/bin:$PATH make fem OMP=true MPI=true PARTS=METIS METISDIR=/usr

# PETSc solver (builds both SPARSKIT and PETSc binaries)
make petsc PETSC_DIR=/usr/lib/petscdir/petsc3.19/x86_64-linux-gnu-real

# Production speed
make fem COMPILER_PROFILE=SPEED

# PGO two-pass
make fem COMPILER_PROFILE=SPEED_PGO_GEN  # step 1: instrument
# ... run training workload ...
make clean                               # step 2: preserves .gcda
make fem COMPILER_PROFILE=SPEED_PGO_USE   # step 3: optimize
```

---

14. Solver Infrastructure

Available Solvers

Solver	Key	Type	Status
SPARSKIT	SPARSKIT	Iterative (BCGstab + ILUT)	**Default, production**
Gauss	GAUSS	Direct band elimination	Legacy
PETSc	PETSC	Iterative (GMRES + BJ)	Fixed, verified
Pardiso	PARDISO	Direct sparse (Intel MKL)	Intel-only
Paralution	PARALUTION	GPU-accelerated iterative	Experimental

All solvers implement the same interface (`system_initialize`, `system_init`, `system_assemble`, `system_solve`, `system_get`).

SPARSKIT (Default)

BCGstab (`itsol=3`) with ILUT preconditioning (`itpre=2`), fill factor `lfil=5`, drop tolerance `1e-2`. Working arrays allocated/deallocated every solve call (~5 MB per timestep for 10k-node mesh). No factorization reuse.

PETSc

GMRES (`KSPGMRES`) with block Jacobi preconditioner (`PCBJACOBI`), rtol=1e-8. Requires global node renumbering for contiguous row ownership. Uses Fortran 2003 OOP (`petsc_system` type with type-bound procedures).

Benchmark (mar_menor): PETSc 2.06s vs SPARSKIT 0.92s. PETSc benefits expected at 100k+ nodes.

---

15. Code Quality Metrics

Allocation Error Handling (`stat=`)

Scope	Total `allocate(`	With `stat=`	Coverage
`src/shyfem/`	727	556	76%
All `src/`	~2,800	~1,709	~61%

Modern File I/O (`newunit=`)

Scope	Total `open(`	With `newunit=`	Coverage
`src/shyfem/`	98	64	65%
All `src/`	~538	~125	~23%

Other Metrics

Metric	`src/shyfem/`	All `src/`
`implicit none` coverage	99.3% (133/134 files)	High
GOTO statements	**0**	403 (89 files, mostly external/tools)
`stop` vs `shympi_abort_internal`	592 stops / 17 abort_internal	See note

The 592 `stop` calls in `src/shyfem/` are primarily allocation failure guards and input validation. They terminate cleanly in serial and hybrid builds. Critical MPI-path stops were converted in Phase 2.

Source File Counts

Location	Files
`src/shyfem/` (main physics)	134 `.f90`

Location	Files
`src/utils/` (utilities)	~250
`src/tools/` (post-processing)	~100
`src/external/` (SPARSKIT + GOTM)	~85
`src/contrib/` (ecological)	~80
`src/obsolete/` (archived)	~400
Active total	**~600 `.f90` files**

---

16. Observations and Improvement Opportunities

Strengths

1. **Clean MPI/serial separation**: `mpi_dummy.f90` provides identical interface – no `#ifdef` guards in physics code
2. **Weighted METIS partitioning**: `vwgt = ilhkv(k)` gives better load balance for 3D baroclinic simulations
3. **Graph-coloring subset decomposition**: Enables OMP tasks in FEM assembly without write conflicts
4. **Allocation caching**: `allocatable, save` pattern avoids ~14 MB/timestep of allocation churn
5. **getpar caching**: `icall=0` pattern eliminates hash lookups in hot paths
6. **Zero GOTOs in main physics**: Complete modernization of `src/shyfem/`
7. **SPARSKIT solver per-rank independence**: No MPI parallelism needed for the solve itself

Known Limitations

1. **GOTM thread-unsafety**: Module-level scratch arrays prevent OMP parallelization of the turbulence node loop. Single largest OMP scalability bottleneck.
2. **`lgr_mpi_global_to_local_elem` is O(nel) linear scan**: Called once per received particle per timestep. A sorted binary search using `ip_sort_elem` would reduce to O(log nel).
3. **Triple-point exchange is O(itrtot) serial point-to-point messages**: Could be batched into collective operations per domain pair.
4. **SPARSKIT allocates/deallocates every timestep**: ~5 MB of working arrays. Caching the ILU factorization across timesteps would help if matrix structure is constant.
5. **Element scatter-add loops remain serial**: `setdepth` element loop, `hydro_vertical` flux accumulation, `current_bottom_stress` – all require subset-based parallelization (complex, needs production-mesh profiling to justify).
6. **`open(unit=...)` holdouts**: ~34 file opens in `src/shyfem/` still use fixed unit numbers, risking unit conflicts.
7. **PETSc bare stops**: `mod_petsc_system.f90` and `mod_petsc_global.f90` contain bare `stop` statements that should use `shympi_abort_internal` in MPI code paths.
8. **`lgr_mpi_exchange` uses `include 'mpif.h'`**: Inconsistent with rest of infrastructure which uses `use mpi` via the shympi module. Prevents compiler type-checking of MPI arguments.
9. **Ecology modules have no feedback to hydrodynamics**: WEUTRO, AQUABC, Mercury are all one-way consumers. No phytoplankton shading, no biogenic turbidity, no oxygen-dependent sediment chemistry. This limits accuracy for eutrophication studies where biological processes significantly affect light penetration and stratification.
10. **Ice model is thermodynamic only**: No ice advection, binary cover (0/1), not thread-safe. Limits applicability to regions where ice dynamics (drift, ridging, lead opening) are important.

11. **z0bk roughness feedback is disabled**: The SEDTRANS05 kernel computes dynamic bed roughness but the feedback path to Chezy bottom friction (`ireib=8`) is commented out. Re-enabling this would close the sediment-friction coupling loop.
12. **Lagrangian particle O(nel) element lookup**: `lgr_mpi_global_to_local_elem` performs a linear scan of the element array for each received particle. Binary search via `ip_sort_elem` would be O(log nel).
13. **BFM (ERSEM) is stub only**: Interface code exists but the actual biogeochemistry library is not linked, limiting advanced ecosystem modeling capabilities.

Improvement Opportunities Ranked by Impact

Rank	Item	Impact	Effort	Category
1	GOTM thread-safety restructuring	High	High	Performance
2	Re-enable z0bk roughness feedback	High	Low	Physics
3	Ecology → hydro feedback (shading/turbidity)	High	Medium	Physics
4	Lagrangian element lookup O(log n)	Medium	Low	Performance
5	SPARSKIT factorization caching	Medium	Medium	Performance
6	Element scatter-add OMP (subset-based)	Medium	High	Performance
7	Ice advection (thermodynamic → dynamic)	Medium	High	Physics
8	BFM library integration	Medium	Medium	Feature
9	Triple-point exchange batching	Low	Medium	Performance
10	`include 'mpif.h'` → `use mpi`	Low	Low	Code quality
11	Remaining `open(unit=...)` → `newunit=`	Low	Low	Code quality
12	PETSc bare stops → `shympi_abort_internal`	Low	Low	Code quality
13	Ice model thread-safety	Low	Medium	Performance

Optimization Status

All easy/moderate OMP wins exhausted (Phases 3-9 complete). Remaining targets are element-to-node scatter-add loops requiring subset-based parallelization – complex and needs production-mesh

profiling (50+ levels, 100k+ elements) to justify the effort.

Benchmarks (mar_menor test case, small mesh)

Configuration	Time
Serial	2.78s
MPI 4P / 1T	1.70s
MPI 4P / 2T	1.67s
MPI 8P	1.86s
SPARSKIT solver	0.92s
PETSc solver	2.06s

Significant scaling gains expected on production meshes with 100k+ nodes and 50+ vertical levels.